BÁO CÁO DỰ ÁN KHOA HỌC KỸ THUẬT: SAFEWEB

Ứng dụng Trí tuệ Nhân tạo (Machine Learning) trong nhận diện rủi ro từ URL

Mục lục

- Chương 1. Giới thiệu tổng quan về SafeWeb
- Chương 2. Lý do chọn đề tài
- Chương 3. Mục tiêu, phạm vi và phương pháp nghiên cứu
- Chương 4. Cơ sở lý thuyết
- Chương 5. Tính mới của đề tài và tiêu chí đánh giá URL
- Chương 6. Dữ liệu và quy trình chuẩn bị dữ liệu
- Chương 7. Quy trình xây dựng mô hình Machine Learning
- Chương 8. Kiến trúc hệ thống và cách SafeWeb hoạt động
- Chương 9. Kiểm thử và kiểm chứng tính đúng đắn
- Chương 10. Kết quả thực nghiệm và phân tích
- Chương 11. Ưu điểm, hạn chế và rủi ro sử dụng
- Chương 12. Hướng phát triển
- Chương 13. Khả năng tái hiện và minh bạch nghiên cứu
- Chương 14. Kết luận
- Phụ lục. Bảng thuật ngữ

Chương 1. Giới thiệu tổng quan về SafeWeb

SafeWeb là gì? SafeWeb là một nguyên mẫu (prototype) phần mềm ứng dụng học máy (Machine Learning) được thiết kế để phân tích các đặc trưng của chuỗi địa chỉ web (URL) nhằm ước tính xem URL đó có mang các dấu hiệu giống với các trang web lừa đảo (phishing) hay là một trang web hợp lệ

(benign). Đây là một dự án mang tính giáo dục và thử nghiệm triển khai kiến trúc tĩnh trên trình duyệt.

Đầu vào và Xử lý: Đầu vào của SafeWeb chỉ đơn thuần là một chuỗi văn bản (text) URL do người dùng cung cấp (ví dụ: `https://example.com/login`). Hệ thống không truy cập, không tải nội dung trang web đích, và không nhấp vào các liên kết chuyển hướng. Thay vào đó, SafeWeb phân tích cấu trúc của chuỗi URL này (như độ dài, các ký tự đặc biệt, từ khóa nghi ngờ) để tạo ra các đặc trưng số học. Sau đó, mô hình học máy sẽ xử lý các đặc trưng này.

Kết quả đầu ra: Hệ thống trả về một tín hiệu đánh giá rủi ro ban đầu (thường được chia thành các mức rủi ro như Thấp, Trung bình, Cao) kèm theo một vài giải thích dựa trên các đặc trưng đã tìm thấy. Kết quả này **không phải** là một bằng chứng tuyệt đối khẳng định trang web đó là an toàn hay độc hại. Nó chỉ mang ý nghĩa cảnh báo rủi ro dựa trên cấu trúc văn bản của URL.

Quy trình hoạt động cơ bản: 1. Người dùng dán một chuỗi URL vào giao diện web. 2. Hệ thống tĩnh trên trình duyệt kiểm tra định dạng URL. 3. JavaScript trích xuất các đặc điểm (features) của chuỗi URL. 4. Mô hình Machine Learning (đã được xuất ra để chạy cục bộ trên trình duyệt) dự đoán mức độ rủi ro và trả về kết quả ngay lập tức mà không cần gửi URL đi bất cứ đâu.

Ví dụ thực tế: Một địa chỉ URL thường gồm 3 phần: Giao thức (`https://`), Tên miền (`facebook.com`), và Đường dẫn (`/login`). Phishing là hành vi kẻ xấu tạo ra tên miền giả mạo (ví dụ: `faceb00k-login.com`) để lừa người dùng nhập mật khẩu. Một URL dài hoặc chứa từ "login" không chắc chắn là lừa đảo (ví dụ trang đăng nhập hợp lệ), nhưng nếu nó kết hợp với một tên miền lạ và nhiều dấu gạch nối, rủi ro sẽ cao hơn.

Chương 2. Lý do chọn đề tài

Nguy cơ lừa đảo trực tuyến (Phishing): Ngày nay, các cuộc tấn công phishing ngày càng tinh vi. Kẻ xấu thường tạo ra các đường link (URL) nhìn lướt qua rất giống với các dịch vụ uy tín (như ngân hàng, mạng xã hội, ví điện tử) để đánh cắp thông tin cá nhân. Bằng mắt thường, người dùng rất khó nhận diện được những khác biệt nhỏ trong cấu trúc URL (ví dụ: dùng chữ `l` thay vì chữ `I`, hoặc thêm các tiền tố/hậu tố làm rối mắt).

Vì sao nhận diện bằng từ khóa là chưa đủ? Nếu chỉ tạo ra một danh sách các từ khóa cố định (như cấm từ "login", "verify") thì hệ thống sẽ chặn nhầm rất nhiều trang web hợp lệ của các ngân hàng hay công ty lớn. Ngược lại, kẻ tấn công cũng dễ dàng lách luật bằng cách đổi từ khóa. Do đó, cần phải quan sát **đồng thời nhiều đặc điểm** của URL (tổng hợp từ độ dài, ký tự, cấu trúc tên miền) để đưa ra quyết định.

Sức mạnh của Machine Learning: Học máy (Machine Learning) là phương pháp lý tưởng cho bài toán này vì nó có khả năng tự động tìm ra các quy luật phức tạp ẩn chứa bên trong hàng ngàn ví dụ (URL độc hại và URL hợp lệ) đã được dán nhãn trước đó, thay vì con người phải lập trình từng quy tắc if-else thủ công.

Việc xây dựng SafeWeb không nhằm mục đích thay thế các phần mềm diệt virus thương mại, mà mang lại giá trị học tập lớn: từ khâu xử lý dữ liệu, huấn luyện mô hình, cho đến việc giải quyết bài toán đưa mô hình AI chạy tĩnh trực tiếp trên trình duyệt để bảo vệ quyền riêng tư.

Chương 3. Mục tiêu, phạm vi và phương pháp nghiên cứu

Mục tiêu chính: Xây dựng một nguyên mẫu ứng dụng học máy có khả năng phân loại rủi ro của URL (hợp lệ hoặc lừa đảo) và triển khai thành công mô hình này chạy trực tiếp trên phía trình duyệt (client-side) mà không cần giao tiếp với máy chủ backend.

Mục tiêu thành phần: - Tiền xử lý và chuẩn hóa tập dữ liệu URL. - Trích xuất các đặc trưng số học từ chuỗi văn bản URL. - Huấn luyện và đánh giá mô hình Random Forest. - Xuất mô hình từ Python sang định dạng JavaScript. - Đảm bảo tính nhất quán (parity) về kết quả dự đoán giữa Python và trình duyệt.

Phạm vi dữ liệu và chức năng: Dự án được xây dựng và kiểm chứng dựa trên tập dữ liệu gồm **5.000 URL** (đã được cân bằng 2.500 lừa đảo, 2.500 hợp lệ) tương thích với chuẩn PhishVN. Chức năng hệ thống dừng lại ở việc **phân tích tĩnh chuỗi văn bản URL**.

Ngoài phạm vi dự án: Dự án **KHÔNG** mở liên kết, **KHÔNG** phân tích mã nguồn HTML/JavaScript của trang đích, **KHÔNG** tra cứu cơ sở dữ liệu danh

sách đen (Blacklist), và **KHÔNG** kiểm tra thông tin máy chủ (DNS, IP, chứng chỉ SSL).

Phương pháp nghiên cứu: Phương pháp thực nghiệm nhị phân (Binary Classification). Sử dụng tập dữ liệu có sẵn để trích xuất đặc trưng, chia tập huấn luyện/kiểm thử theo chiến lược tách biệt tên miền (domain-disjoint) để giảm thiểu rò rỉ dữ liệu, và lựa chọn mô hình dựa trên các chỉ số đánh giá tiêu chuẩn.

Chương 4. Cơ sở lý thuyết

- **URL (Uniform Resource Locator):** Địa chỉ định vị tài nguyên trên Internet, bao gồm các thành phần: giao thức (http/https), tên miền (domain), cổng (port), đường dẫn (path) và tham số (query).
- **Phishing (Lừa đảo trực tuyến):** Kỹ thuật giả mạo các thực thể uy tín qua email hoặc tin nhắn để lừa người dùng nhấp vào URL độc hại nhằm đánh cắp thông tin.
- **Phân loại nhị phân (Binary Classification):** Bài toán học máy yêu cầu phân loại dữ liệu vào một trong hai nhóm. Trong dự án này là: Benign (Hợp lệ) và Phishing (Lừa đảo).
- **Machine Learning (Học máy):** Lĩnh vực trí tuệ nhân tạo cho phép máy tính tự học các mẫu (patterns) từ dữ liệu thay vì được lập trình quy tắc cụ thể.
- **Dữ liệu huấn luyện (Training Set) và Dữ liệu kiểm thử (Test Set):** Dữ liệu huấn luyện dùng để dạy mô hình; dữ liệu kiểm thử dùng để kiểm tra xem mô hình áp dụng kiến thức đã học lên dữ liệu mới có tốt hay không.
- **Đặc trưng (Features) và Nhãn (Labels):** Nhãn là kết quả đã biết (phishing=1, benign=0). Đặc trưng là các đặc điểm được trích xuất từ URL (ví dụ: độ dài URL = 45 ký tự) để làm đầu vào cho mô hình.
- **Random Forest:** Thuật toán học máy hoạt động bằng cách tạo ra một "rừng" gồm nhiều "cây quyết định" (Decision Trees). Mỗi cây sẽ tự xem xét một tập hợp các đặc trưng ngẫu nhiên để đưa ra dự đoán. Kết quả cuối cùng được bình chọn từ tất cả các cây. Nhờ kết hợp nhiều cây, Random Forest giảm thiểu sai số và hoạt động ổn định.

- **Sự sai số của mô hình:** Mô hình học từ các ví dụ trong quá khứ. Do đó, với những URL sử dụng kỹ thuật ngụy trang hoàn toàn mới mà nó chưa từng gặp, mô hình vẫn có thể dự đoán sai. Kết quả tốt trong thực nghiệm không đảm bảo sự hoàn hảo 100% trên mạng lưới Internet rộng lớn.

Chương 5. Tính mới của đề tài và tiêu chí đánh giá URL

5.1. Tính mới của đề tài

Tính mới của SafeWeb không nằm ở việc phát minh ra một thuật toán học máy mới, mà nằm ở **cách thức triển khai và tích hợp kiến trúc**: - Chuyển đổi toàn bộ logic nội suy của mô hình Machine Learning (Random Forest) từ môi trường máy chủ (Python) sang môi trường trình duyệt (JavaScript). - Tạo ra một ứng dụng tĩnh (Static Site trên GitHub Pages) có khả năng phân tích rủi ro URL cục bộ ngay trên máy người dùng mà không cần backend API, giúp bảo vệ tối đa quyền riêng tư (URL không bị gửi đi). - Có quy trình đối chiếu tự động đảm bảo kết quả tính toán giữa Python và JavaScript hoàn toàn khớp nhau (Parity Check).

5.2. Các tiêu chí đánh giá URL

Hệ thống sử dụng mã nguồn để trích xuất các đặc trưng sau từ chuỗi URL: - **Kích thước:** Độ dài tổng thể của URL, độ dài của tên miền (domain). - **Ký tự:** Đếm số lượng dấu chấm (.), dấu gạch chéo (/), dấu gạch nối (-), dấu gạch dưới (_), số lượng chữ số và các ký tự đặc biệt. Lừa đảo thường dùng nhiều dấu gạch nối để tạo tên miền giả. - **Cấu trúc mạng:** Kiểm tra URL có sử dụng giao thức HTTPS hay không (`has_https`), có sử dụng thẳng địa chỉ IP làm tên miền hay không (`has_ip`), và đếm số lượng tên miền phụ (subdomains). Phishing thường dùng nhiều subdomains để che giấu tên miền gốc. - **Dấu hiệu đáng ngờ:** Sự xuất hiện của ký tự @, ?, =, % và các từ khóa nhạy cảm (như `login`, `verify`, `account`, `bank`). - **Độ phức tạp (Entropy):** Đo lường sự lộn xộn của chuỗi ký tự. URL lừa đảo được tạo tự động thường có độ lộn xộn cao.

Lưu ý quan trọng: Không một tiêu chí nào đứng riêng lẻ đủ để kết luận URL là lừa đảo. Một URL chứa từ "login" (ví dụ: `https://github.com/login`) hoàn toàn an toàn. Hệ thống phải tổng hợp trọng số của tất cả các tiêu chí này

thông qua mô hình học máy. Vì mã nguồn **không** kiểm tra chứng chỉ SSL hay tra cứu DNS, hệ thống chỉ hoạt động dựa trên các tín hiệu văn bản thu thập được.

Chương 6. Dữ liệu và quy trình chuẩn bị dữ liệu

Theo bằng chứng dự án (ML_EXPERIMENT_REPORT), tập dữ liệu cuối cùng được sử dụng là tập URL tương thích với định dạng PhishVN, gồm **5.000 bản ghi**. - **Số lượng:** 2.500 URL hợp lệ (benign) và 2.500 URL lừa đảo (phishing). - **Phân loại tier:** Gồm 2.119 nhãn Gold, 2.457 Silver, và 424 Bronze (mức độ tin cậy của nguồn theo quy ước của dữ liệu gốc).

Quy trình làm sạch và kết quả Audit (Kiểm toán): Script `scripts/prepare_phishvn_dataset.py` được sử dụng để chuẩn hóa dữ liệu. Kết quả kiểm toán được ghi nhận: - 0 bản ghi trùng lặp chính xác. - 0 bản ghi gần trùng sau khi chuẩn hóa. - 1 tên miền trùng lặp được phát hiện trong phần tóm tắt. - 87 họ hostname (hostname-family) bị phát hiện có sự chồng lấn trong bài kiểm tra sâu hơn. - 498 URL sử dụng HTTPS; 0 URL sử dụng IP trực tiếp làm host. - 6 URL sử dụng chuẩn Punycode/IDN. - 315 URL chứa từ khóa đáng ngờ. - 2.498 URL phishing dạng ngắn. - 0 URL benign dạng dài.

Tập dữ liệu này đã được chuẩn hóa và cân bằng tốt cho việc thử nghiệm, nhưng việc thiếu vắng các URL benign dạng dài hoặc URL host IP cho thấy nó vẫn chưa đại diện hoàn hảo cho mọi cấu trúc URL trên Internet. Sự chồng lấn 87 họ hostname cũng cho thấy rủi ro rò rỉ dữ liệu (data leakage) chưa được loại bỏ tuyệt đối 100%.

Chương 7. Quy trình xây dựng mô hình Machine Learning

Quy trình xây dựng mô hình SafeWeb đi qua các bước sau:

1. **Tiếp nhận & Chuẩn hóa:** Nạp 5.000 URL, làm sạch các bản ghi lỗi và loại bỏ trùng lặp.
2. **Trích xuất đặc trưng (Feature Extraction):** Chuyển đổi các URL dạng văn bản thành các ma trận số học dựa trên các tiêu chí (độ dài, ký tự, entropy) đã định nghĩa ở Chương 5.

3. **Phân chia dữ liệu (Train/Test Split):** Thay vì chia dữ liệu ngẫu nhiên (dễ dẫn đến việc mô hình "học thuộc" tên miền), dự án áp dụng chiến lược **tách biệt tên miền (domain-disjoint)**. Nghĩa là, nếu một tên miền xuất hiện trong tập huấn luyện, các URL thuộc tên miền đó sẽ bị hạn chế xuất hiện trong tập kiểm thử. Điều này bắt buộc mô hình phải học các quy luật chung thay vì học vẹt tên miền.
4. **Huấn luyện mô hình:** Thử nghiệm đưa dữ liệu vào hai thuật toán là Random Forest và Gradient Boosting.
5. **Đánh giá và Lựa chọn:** Mô hình Random Forest cho kết quả cân bằng và ổn định nhất, đồng thời kiến trúc dạng "cây" của nó hỗ trợ việc chuyển đổi sang JavaScript dễ dàng hơn so với Gradient Boosting. Do đó, Random Forest được chọn làm mô hình chính thức.
6. **Xuất mô hình:** Sử dụng script để chuyển đổi tệp mô hình của Python (`random_forest.joblib`) sang tệp dữ liệu tĩnh cho trình duyệt (`model-data.js`).

Chương 8. Kiến trúc hệ thống và cách SafeWeb hoạt động

Thành phần công nghệ: - **Python:** Ngôn ngữ dùng ở hậu trường (backend/ local) để xử lý dữ liệu, huấn luyện mô hình và chạy các script kiểm thử, đánh giá. - **Random Forest (Scikit-learn):** Thuật toán học máy cốt lõi tạo ra các cây quyết định. - **JavaScript:** Ngôn ngữ xử lý logic trên giao diện người dùng. Đảm nhiệm việc trích xuất đặc trưng và chạy nội suy mô hình (browser inference). - **GitHub Pages:** Nền tảng hosting dùng để triển khai giao diện người dùng dưới dạng một trang web tĩnh (Static Site). - **Flask:** Một framework web của Python, chỉ được lưu lại trong mã nguồn nhằm phục vụ việc phát triển và kiểm thử cục bộ, **không** được sử dụng trong bản phát hành tĩnh trên GitHub Pages.

Sơ đồ luồng hoạt động trên trình duyệt: `Người dùng nhập URL` -> `Kiểm tra định dạng (JavaScript)` -> `Trích xuất đặc trưng không cần mạng (JavaScript)` -> `Truy xuất dữ liệu cây quyết định từ model-data.js` -> `Tính toán điểm rủi ro` -> `Hiển thị cảnh báo & Giải thích trên giao diện`.

Điểm nổi bật của kiến trúc này là quy trình tính toán được thực hiện hoàn toàn trên máy khách (Client-side), tiết kiệm chi phí máy chủ và bảo mật thông tin truy vấn của người dùng.

Chương 9. Kiểm thử và kiểm chứng tính đúng đắn

Dự án đã thiết lập các bộ kiểm thử tự động để đảm bảo tính ổn định:

1. **Kiểm thử Python (Pytest):** Bằng chứng ghi nhận `39 passed in 5.96s`. Giúp đảm bảo quy trình trích xuất đặc trưng và làm sạch dữ liệu phía Python không bị lỗi.
2. **Kiểm thử trình duyệt (Browser Tests):** Bằng chứng ghi nhận 6 `passed, 0 failed` (`npm run test:browser`). Kiểm tra việc JavaScript xử lý đúng các URL lạ, từ chối URL sai định dạng, và **xác nhận client không bao giờ fetch() hay gửi URL ra ngoài Internet**.
3. **Kiểm tra tính nhất quán (Parity Check):** Đây là bài kiểm tra quan trọng nhất (`compare_browser_model.py`). Bằng chứng ghi nhận đã thử nghiệm trên 10 URL hợp lệ và 6 URL không hợp lệ.
4. Kết quả: Độ lệch đặc trưng tối đa (max feature delta) chỉ là 8.88×10^{-16}.
5. Độ lệch xác suất tối đa (max probability delta): `0`.
6. Điều này chứng minh thuật toán tính toán bằng JavaScript trên trình duyệt hoạt động chính xác tuyệt đối 100% so với mô hình gốc trên Python.
7. **Kiểm thử tổng thể:** Lệnh `npm test` ghi nhận `48 passed in 12.83s`.

(Lưu ý: Các bài kiểm thử này dùng để chứng minh mã nguồn không bị lỗi (bugs) và kiến trúc hoạt động đúng thiết kế, nó không phải là kiểm thử chứng minh khả năng phát hiện lừa đảo trong thực tế).

Chương 10. Kết quả thực nghiệm và phân tích

Kết quả đánh giá mô hình Random Forest trên tập kiểm thử bằng phương pháp tách biệt tên miền (domain-disjoint) được ghi nhận trong báo cáo thực nghiệm (ML_EXPERIMENT_REPORT) như sau:

- **Accuracy (Độ chính xác): 0,751 (75,1%)** - Trong toàn bộ các URL được đưa vào tập kiểm thử, mô hình dự đoán đúng 75,1% số lần.
- **Precision (Độ chuẩn xác): 0,765 (76,5%)** - Trong số những URL mà mô hình "gắn cờ" là lừa đảo, có 76,5% thực sự là lừa đảo (tỷ lệ báo động nhầm - False Positive là khoảng 23,5%).
- **Recall (Độ bao phủ): 0,769 (76,9%)** - Trong toàn bộ các URL lừa đảo thực sự có trong tập kiểm thử, mô hình đã "bắt" được 76,9% (bỏ lọt khoảng 23,1%).
- **F1-Score: 0,767** - Điểm trung bình điều hòa, cho thấy mô hình có sự cân bằng tương đối tốt giữa việc bắt lừa đảo và hạn chế báo động nhầm.

Phân tích kết quả: Việc chỉ đạt Accuracy 75,1% (thay vì 100% như trên các bộ dữ liệu demo đồ chơi trước đây) là một kết quả thực tế và phản ánh đúng độ khó của bài toán nhận diện phishing.

Vì sao mô hình đoán sai? - **Bỏ lọt (False Negatives):** Các kiểm thử Parity ghi nhận một số URL có dấu hiệu đáng ngờ (như nhiều subdomain, dùng host IP, dùng dịch vụ rút gọn link) vẫn bị dự đoán là `legitimate`. Lý do là kẻ tấn công ngày càng ngụy trang tên miền trông rất "sạch" và giống với tên miền hợp lệ. - **Báo động nhầm (False Positives):** Các URL hoàn toàn hợp pháp nhưng có cấu trúc dài, tham số phức tạp, hoặc vô tình chứa từ khóa như `login` có thể bị mô hình nhầm lẫn là phishing do quá trình học máy thiết lập trọng số nhạy cảm với các yếu tố này.

Chương 11. Ưu điểm, hạn chế và rủi ro sử dụng

Ưu điểm: - Kiến trúc sáng tạo: Đưa mô hình học máy chạy tĩnh trên trình duyệt giúp ứng dụng phản hồi nhanh, không cần bảo trì máy chủ và bảo vệ quyền riêng tư. - Phương pháp luận rõ ràng: Dự án áp dụng phương pháp domain-disjoint để đánh giá khắt khe mô hình, có quy trình Parity Check để đảm bảo tính nhất quán công nghệ.

Hạn chế: - **Dữ liệu hạn chế:** Mặc dù đã chuẩn hóa 5.000 URL, tập dữ liệu này chưa bao phủ hết các dạng URL thực tế (thiếu IP host, thiếu benign dài). Sự chồng lấn 87 họ tên miền cho thấy nguy cơ rò rỉ dữ liệu huấn luyện vẫn tồn tại. - **Đặc trưng nông (Shallow features):** Hệ thống chỉ phân tích cấu trúc chuỗi văn bản. Nó hoàn toàn "mù" trước nội dung thực sự của trang web, không kiểm tra chứng chỉ SSL, và không tra cứu danh tiếng tên miền.

Rủi ro sử dụng: - **Không thay thế phần mềm bảo mật:** Kết quả của SafeWeb chỉ là tín hiệu tham khảo. Nếu người dùng hoàn toàn tin tưởng vào kết quả "Rủi ro thấp" để nhập mật khẩu vào một trang web, họ vẫn có thể bị tấn công bởi những chiến dịch phishing tinh vi sử dụng URL sạch. Dự án chỉ mang tính nguyên mẫu nghiên cứu.

Chương 12. Hướng phát triển

Để phát triển SafeWeb từ một nguyên mẫu giáo dục thành một công cụ thực tiễn hơn, các hướng đi tiếp theo bao gồm: 1. **Nâng cấp Dữ liệu:** Thu thập tập dữ liệu lớn hơn, đa dạng hơn từ các nguồn tình báo mối đe dọa (Threat Intelligence) thực tế, và khắc phục triệt để sự chồng lấn tên miền khi chia tập. 2. **Tích hợp Đặc trưng Động (Dynamic Features):** Bổ sung các module backend hoặc API ngoại tuyến để kiểm tra tuổi của tên miền (WHOIS), chứng chỉ bảo mật, và đối chiếu với các danh sách đen (Blacklists) cộng đồng. 3. **Cải thiện độ giải thích (Explainability):** Hiển thị rõ ràng cho người dùng biết chính xác những đặc trưng nào (ví dụ từ khóa nào, độ dài bao nhiêu) đóng góp lớn nhất vào quyết định "Rủi ro Cao" của hệ thống, sử dụng các kỹ thuật như SHAP. 4. **Phát triển Tiện ích mở rộng (Browser Extension):** Đóng gói logic hiện tại thành một tiện ích mở rộng trên Chrome/Edge để tự động cảnh báo người dùng trước khi họ nhấp vào các liên kết trong email.

Chương 13. Khả năng tái hiện và minh bạch nghiên cứu

Dự án duy trì mức độ minh bạch cao, cho phép bất kỳ ai có kiến thức cơ bản về Python và Node.js đều có thể tái hiện toàn bộ quy trình từ xử lý dữ liệu đến triển khai mô hình.

Các tập lệnh (scripts) trọng tâm đảm bảo tính tái hiện bao gồm: - Tiền xử lý dữ liệu: `scripts/prepare_phishvn_dataset.py` - Huấn luyện mô hình: `scripts/train_phishvn_model.py` (Lưu mô hình tại `model/random_forest.joblib`). - Xuất mô hình sang JS: `scripts/export_browser_model.py` (Tạo ra tệp `web/static/model-data.js` có dung lượng khoảng 7.2MB). - Kiểm tra tính nhất quán: `scripts/compare_browser_model.py`. - Lệnh chạy kiểm thử phần mềm: `npm test` và `pytest`.

Môi trường yêu cầu cơ bản là Python 3.11+ và Node.js. Mã nguồn, script chia tập và quy trình đánh giá được giữ công khai trong kho mã (repository) để cộng đồng có thể giám sát và kiểm chứng các số liệu thực nghiệm.

Chương 14. Kết luận

SafeWeb đã hoàn thành xuất sắc mục tiêu xây dựng một nguyên mẫu giáo dục về ứng dụng học máy trong bảo mật. Dự án chứng minh tính khả thi của việc thiết kế một quy trình đồng bộ: từ chuẩn hóa 5.000 URL, huấn luyện mô hình Random Forest, cho đến việc chuyển đổi mô hình thành công sang môi trường JavaScript.

Việc ứng dụng hoạt động hoàn toàn dưới dạng tĩnh (Static Site) trên trình duyệt, không cần giao tiếp máy chủ, là một điểm sáng về kiến trúc phần mềm, đảm bảo hiệu năng và quyền riêng tư. Với độ chính xác thực nghiệm đạt 75,1% trong điều kiện chia tập domain-disjoint khắt khe, mô hình cho thấy khả năng nhận diện tín hiệu lừa đảo cơ bản từ chuỗi văn bản.

Mặc dù vẫn mang các hạn chế cố hữu của việc chỉ phân tích cú pháp tĩnh và chưa thể thay thế các hệ thống an toàn thông tin chuyên nghiệp, SafeWeb cung cấp một nền tảng vững chắc và minh bạch để tiếp tục phát triển, nghiên cứu sâu hơn về các đặc trưng chống phishing trong tương lai.

Phụ lục. Bảng thuật ngữ

- **URL (Uniform Resource Locator):** Địa chỉ truy cập một tài nguyên cụ thể trên mạng Internet.

- **Phishing:** Hình thức lừa đảo trực tuyến bằng cách giả mạo các trang web uy tín để đánh cắp thông tin cá nhân.
- **Benign:** Trong ngữ cảnh này nghĩa là hợp lệ, an toàn, không có mục đích độc hại.
- **Feature (Đặc trưng):** Một thuộc tính hoặc đặc điểm đo lường được từ dữ liệu (VD: độ dài URL) dùng làm đầu vào cho mô hình học máy.
- **Label (Nhãn):** Đáp án hoặc phân loại thực sự của một mẩu dữ liệu (VD: 1 = Phishing, 0 = Benign).
- **Dataset (Tập dữ liệu):** Tập hợp các dữ liệu đã được thu thập để huấn luyện và kiểm tra.
- **Training Set / Test Set:** Dữ liệu huấn luyện dùng để dạy cho AI; Dữ liệu kiểm thử dùng để chấm điểm AI sau khi học.
- **Random Forest:** Thuật toán học máy dựa trên việc xây dựng và kết hợp kết quả dự đoán của nhiều cây quyết định.
- **Accuracy / Precision / Recall / F1-Score:** Các chỉ số toán học dùng để đo lường độ chính xác và khả năng bao quát của mô hình phân loại.
- **Domain-disjoint:** Chiến lược chia tách dữ liệu sao cho các tên miền có mặt trong tập huấn luyện sẽ không xuất hiện trong tập kiểm thử, giúp đánh giá khả năng tổng quát hóa của mô hình.
- **False Positive (Báo động nhầm):** Khi hệ thống đánh giá một URL an toàn là lừa đảo.
- **False Negative (Bỏ lọt):** Khi hệ thống đánh giá một URL lừa đảo là an toàn.
- **Browser Inference:** Quá trình sử dụng mô hình học máy để suy luận và đưa ra dự đoán trực tiếp bằng tài nguyên của trình duyệt web.
- **Flask:** Một công cụ (framework) lập trình web bằng Python dùng ở máy chủ.
- **GitHub Pages:** Dịch vụ lưu trữ miễn phí của GitHub để xuất bản các trang web tĩnh (chỉ có HTML/CSS/JS).